What is Instance Variable OR Non static field :

* It is a variable which we declare at **class level so, we have default value.**
* If we declare a variable **inside a class but outside of a method without static modifier then it is called non static field.**
* The life of any non static field starts with **Object creation** that remove we concrat thoto about non static field without **object creation**.

Example

```
public class Demo
{
    int x = 100; //non static field

    public static void main(String[] args)
    {
        System.out.println("x value is "+x); //error
    }
}
```

* As far as it's accessibility is concerned, non static fields are accessible **within the same** class as all as depends upon the **access modifier** applied on non static field.
* Whenever we want to represent **object properties** we should use non static field.
* The life (lost scope) of non static field starts with object and if the object is destroyed then automatically non static field will also be deleted from the memory.

How to initialize the object properties through methods :

* If we create our OLC and OLC classes into two different packages then from another package we can access the OLC and OLC classes to initialize the non static field with public access modifier **(Strongly not recommended)**.
So, the alternate way is to initialize the object properties (NSF) is public methods as shown in the program.

//Program :

```
package com.rwd.olc;

import java.util.Scanner;

//OLC
public class Employee
{
    int employeeNumber;
    String employeeName;
    double employeeSalary;

    public void setEmployeeData()
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter employee number: ");
        employeeNumber = sc.nextInt();
        System.out.println("Enter employee name: ");
        employeeName = sc.next();
        System.out.println("Enter employee salary: ");
        employeeSalary = Double.parseDouble(sc.nextline());
        sc.close();
    }

    public void getEmployeeData()
    {
        System.out.println("Employee Number is: "+employeeNumber);
        System.out.println("Employee Name is: "+employeeName);
        System.out.println("Employee Salary is: "+employeeSalary);
    }

}


package com.rwd.olc;

import com.rwd.olc.Employee;

//OLC
public class EmployeeDemo
{
    public static void main(String[] args)
    {
        Employee ccsft = new Employee();
        ccsft.setEmployeeData();
        ccsft.getEmployeeData();
    }
}
```

How to initialize my object properties via methods using parameter variable :

* Instead of initializing the non static field with method without parameters, it is recommended to initialize the non static field **method with parameter as shown in the program.**

```
package com.rwd.olc;

public class Customer
{
    int customerId;
    String customerName;

    public void setCustomerData(int cid, String cname)
    {
        customerId = cid;
        customerName = cname;
    }

    public void getCustomerData()
    {
        System.out.println("Customer ID is: "+customerId);
        System.out.println("Customer Name is: "+customerName);
    }
}

package com.rwd.olc;

import com.rwd.olc.Customer;

public class CustomerDemo
{
    public static void main(String[] args)
    {
        Customer rwj = new Customer();
        rwj.setCustomerData(122, "Raj");
        rwj.getCustomerData();

        System.out.println("-----------");

        Customer priya = new Customer();
        priya.setCustomerData(101, "Priyi");
        priya.getCustomerData();
    }
}
```

Note : Upto here, We know total 3 ways to initialize the object properties which are as follows :

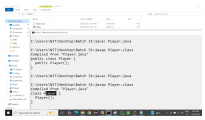
1) Using Object reference(rwj.rollNumber = 122)
2) Using Method without parameter (Scanner class)
3) Using Method with Parameter (Push Style)

Introduction of Constructor : [Introduction only]

* If the name of the class and name of the method, both are exactly same and It does not contain any return type then it is called Constructor.

```
public class Student
{
    public Student() //Constructor
    {
    }
}
```

* IN JAVA, WHENEVER WE WRITE A CLASS AND IF WE DON'T WRITE ANY TYPE OF CONSTRUCTOR IN THE CLASS THEN AUTOMATICALLY javac (java compiler) will add one default no argument constructor in that particular class.

Example

* Every java class must contain **at-least one constructor** either implicitly added by java compiler OR explicitly written by developer. [We cannot think about java class without constructor]
* The access modifier of default no argument constructor, added by java compiler depends upon class access modifier that means if the class is public then the default no argument constructor added by java compiler is also public, if class is not public then constructor is also not public.